SMART STUDY RECALL / ENGINEERING AND RESEARCH GUIDE

RAG Study Assistant Architecture

Document ingestion, passage retrieval, adaptive heuristics, grounded generation and evaluation

Prepared from the repository implementation and experiment reports as inspected on 27 October 2026.

Faculty presentation edition | Implementation facts are separated from research hypotheses and unvalidated claims.

How to use this document

This guide explains the running implementation, what data enters each component, what algorithms do with it, what is saved, how to operate the project, and what the current experiments do and do not establish. Formulae use plain ASCII notation. Source paths at the end let a reviewer trace each description back to the repository.

Important interpretation rule: a predicted next-session correct fraction is a score proxy. It is not a measured probability that a learner will remember a course concept on a future date.

Contents

01. 1. RAG concept and actual architecture

02. 2. Upload, extraction and chunking

03. 3. Standard BM25 retrieval

04. 4. Hybrid TF-IDF and Reciprocal Rank Fusion

05. 5. Recall-aware state reranking

06. 6. Grounded generation, citations and fallback

07. 7. Data, user state and persistence

08. 8. How to evaluate this RAG system

09. 9. Limitations and next iterations

1. RAG concept and actual architecture

Retrieval-augmented generation (RAG) first finds relevant source passages and then asks a language model to answer from those passages. This project stores extracted text chunks in PostgreSQL and retrieves them with token and TF-IDF ranking. It does not use a hosted embedding model or a persistent vector database.

Upload source PDF/DOCX/TXT/MD	Extract and chunk 210 words, 35 overlap	Persist chunks PostgreSQL	Retrieve evidence BM25 / TF-IDF / RRF	Generate or extract Cited output

The term semantic in the UI refers to TF-IDF term vector cosine similarity. TF-IDF is a sparse lexical weighting technique, not a dense neural sentence embedding. The architecture display describes the flow; it is explanatory visualization rather than a live trace of an individual request.

---

2. Upload, extraction and chunking

Allowed formats are PDF, DOCX, TXT and Markdown. PDF text is extracted page-wise using a PDF parser; DOCX paragraphs and table cells are included; text/Markdown decode as UTF-8 with byte-order mark support. Null bytes and extra whitespace are normalized. Encrypted files are rejected. Empty extracted text causes an actionable error; scanned image PDFs need OCR before upload.

A 12 MB upload limit protects the route. Extracted content is capped at 1.2 million characters. The chunker splits on whitespace into windows of 210 words, then advances by 175 words, giving 35 words of overlap. The overlap preserves a small context bridge near chunk boundaries. Up to eight frequent non-stopword keywords are stored for each chunk.

The database stores material metadata, text content, chunk index and keyword array. A file SHA-256 hash detects duplicate uploads by the same uploader. Material access permits published material or the current uploader's own unpublished material; administrators have elevated manage privileges.

---

3. Standard BM25 retrieval

Tokenization extracts Unicode word-like terms, lowercases them, strips edge punctuation, removes terms of length at most two and removes a predefined stopword set. The query and each passage are tokenized. The implementation computes document frequency and average passage length across all chunks at query time.

```
BM25(D,Q) = sum over query terms t of IDF(t) * [ f(t,D) * (k1 + 1) ] / [ f(t,D) + k1 * (1 - b + b *
|D| / avgdl) ]
IDF(t) = ln(1 + (N - df(t) + 0.5) / (df(t) + 0.5))
Project values: k1 = 2.5, b = 0.75
```

Raw BM25 scores are normalized by dividing by the maximum score for this query so the best score is 1. Standard mode sorts by normalized score and chunk index. If the query has no matching BM25 terms it can return no evidence. Its strength is exact term matching and interpretability; it may miss paraphrases and synonyms.

---

4. Hybrid TF-IDF and Reciprocal Rank Fusion

Hybrid retrieval computes BM25 and fits a scikit-learn TfidfVectorizer to the chunk corpus plus the current query. It limits vocabulary to 8,000 features, uses English stop words, sublinear term frequency and unigrams plus bigrams. Cosine similarity ranks each chunk relative to the query. The vectorizer is rebuilt at query time; no dense document vectors are saved.

```
sublinear_tf = 1 + ln(raw_term_frequency)
smoothed_idf = ln((1 + number_of_documents) / (1 + document_frequency)) + 1
tfidf_vector = L2_normalize(sublinear_tf * smoothed_idf)
cosine(q,d) = dot(q,d) / (norm(q) * norm(d))
```

```
RRF(d) = sum over ranked lists L of 1 / (k + rank_L(d))
Project constant k = 60
Two lists: BM25 ranking and TF-IDF cosine ranking
```

RRF fuses rank positions, which avoids directly adding raw scores with different scales. The fused score is normalized relative to the highest RRF score. Stable chunk-index tie breaking makes ordering deterministic. The engine retains at most six chunks for chat; notes/quizzes may select and diversify a larger evidence set.

Hybrid is non-personalized. Standard and hybrid form useful retrieval baselines. Their score magnitudes are relative to the query and ranking method, not calibrated probabilities that a passage is relevant.

---

5. Recall-aware state reranking

Adaptive mode begins with the hybrid retrieval set. The backend reads prior material quiz responses per learner and material. A concept enters the gap map only with at least two responses and accuracy below 80%. Its gap is max(0, 0.8 - concept_accuracy) multiplied by min(response_count/3, 1). This keeps one-off mistakes from dominating and caps evidence weight.

Recency is elapsed time since a concept's most recent quiz, divided by 30 days and clipped to 0..1. It is a time-based heuristic, not a forgetting probability. The reranker matches token overlap between concept name and passage and computes a gap relevance and recency relevance. The final score is base RRF score plus weighted signals.

```
final = RRF_base + gap_weight * intent_scale * gap_relevance
+ recency_weight * intent_scale * recency_relevance
default gap_weight = 0.30; recency_weight = 0.15
intent_scale = 1.0 for review/practice intent; 0.25 for direct factual questions
```

A gap boost may be larger for explicit revision intent. A deterministic keyword intent set checks terms such as review, practice, quiz, forgot and revise. Matched concepts and the state signals can be exposed in diagnostics. This is not a neural reranker and does not infer a hidden mastery state.

The intervention helper can recommend targeted practice for a repeated gap, retrieval practice when a concept has not been quizzed for at least 24 days, a self-test for explicit revision intent, or a baseline quiz when no state exists. These thresholds are transparent heuristics.

---

6. Grounded generation, citations and fallback

The answer helper receives the question, at most eight conversation-history turns and top source excerpts labeled with chunk IDs. When AI_API_KEY and AI_MODEL are configured, the backend calls the selected provider via the configured API-compatible endpoint. For Gemini configuration, it uses Google's OpenAI-compatible Chat Completions endpoint; the default OpenAI-compatible route uses the configured OpenAI endpoint.

The system prompt tells the provider to answer only from excerpts, treat source text as untrusted instructions, abstain if evidence is insufficient and return structured JSON containing answer text and citation IDs. The backend filters

citations to IDs among retrieved chunks. If the generated response is missing a valid answer/citation or structured JSON cannot be parsed, it falls back to extraction from the best retrieved sentences/passages.

When no evidence score reaches the minimum threshold, the chatbot returns an explicit not-found-in-material response with no citation. If the external provider is unconfigured, extractive notes/questions/chat paths are used. Provider secrets stay in backend .env, never in browser JavaScript.

Notes request concise key ideas and citations. Quiz generation asks for short, checkable question/answer pairs based on source excerpts and one allowed source chunk per item. If provider output is unusable, the extractive question fallback blanks a frequent source keyword inside a sentence. Thus not every generated quiz is an MCQ; the current RAG quiz is primarily short-answer/fill-in-the-blank.

---

7. Data, user state and persistence

Documents and chunks are in PostgreSQL. Material quiz sessions and responses also live in the database. Persistent chatbot history is stored in study_rag_interactions after migration 004; rows include learner, material, prompt, answer, retrieval mode/method, citations, diagnostics and an optional yes/no source-support judgment.

Conversation history is limited when sent to the provider for context. The saved history route returns recent interactions for the authenticated learner and material. The RAG setup selection may be browser-local per signed-in user, but the document and chat records are server-side.

Do not confuse user feedback that a source supported an answer with an objective correctness label. It is a subjective proxy. A rigorous benchmark needs reviewed question-passage relevance judgments and expert answer/citation assessment.

---

8. How to evaluate this RAG system

Current Standard/Hybrid/Adaptive options can be compared on a query, but the codebase has no human-judged retrieval dataset or answer-grounding benchmark. Therefore system accuracy, citation validity and learning benefit are not evaluated. A few chats or a high ranking score cannot establish model quality.

Evaluation dimension	Recommended metric	Formula / question
Retrieval recall	Recall@k	Relevant passages in top k / all judged relevant passages
Rank quality	MRR	Mean reciprocal rank of first relevant passage
Rank quality	nDCG@k	Discounted gain in top k normalized by ideal ordering
Answer correctness	Accuracy	Expert-judged correct answers / judged answers
Grounding	Citation precision	Citations that support claim / citations reviewed
Abstention	Unsupported-claim rate	Answers with unsupported substantive claim / answers reviewed
Reliability	Latency distribution	Median and p95 retrieval-only and end-to-end latency

Construct a held-out query set with source chunk relevance labels. Compare BM25, BM25+TF-IDF/RRF, and adaptive reranking on identical documents and queries. Evaluate generation separately with expert judgments for correctness, support, citation validity and abstention. Bootstrap by query or material. Keep learner-state features causal: only

include responses observed before the query.

The comparison UI's retrieval rank score is query-relative, not confidence. Learner support ratings are not ground-truth accuracy. Report 'Not evaluated' until labels exist.

9. Limitations and next iterations

- TF-IDF is lexical and sparse; it does not provide transformer sentence embeddings. It is a lightweight local approach with limited semantic paraphrase handling.
- The vectorizer is rebuilt for each query; this costs time as documents grow. No persistent vector index exists.
- Chunking uses a fixed word window and overlap. It does not preserve full document layout, table structure for every format or OCR images.
- Adaptive signals need repeat quizzes and exact concept labels. Term overlap can miss paraphrases and may boost a passage for a concept that is only lexically present.
- LLM answers remain provider-dependent. Citation-ID filtering enforces citations come from retrieved chunks, not that each cited passage actually supports every claim.

A research roadmap can add dense embeddings only if an evaluated retrieval benchmark shows benefit; add a reviewed corpus and answer audit; version prompts and provider models; store request timing and retrieval diagnostics; and validate accessibility and privacy before using real learner content.

Repository traceability and source files

The report describes the local repository snapshot, not an independently deployed cloud instance. A running deployment can differ if its backend, database migrations, model artifacts or environment variables have not been updated.

Primary source files reviewed:

- app/services/rag_engine.py
- app/services/study_materials.py
- app/routers/materials.py
- app/routers/rag_chatbot.py
- app/models.py
- migrations/003_ai_study_materials.sql
- migrations/004_rag_chat_history.sql
- docs/RESEARCH_LIMITATIONS.md